

Syslab机器学习工具箱

任品

苏州同元软控信息技术有限公司

2025年5月27日



TONGYUAN
Software and Control

未经许可，不得复印、传播或以其他方式使用
@苏州同元软控信息技术有限公司版权所有，侵权必究

课程须知

本课程课程目标:

本课程面向机器学习工具箱新用户，讲述机器学习的基本概念，并且在 Syslab 中演示如何使用工具箱函数求解具体问题，帮助用户对机器学习建立基本了解，并且学会在 Syslab 中使用机器学习工具箱进行机器学习模型的创建、训练、验证。

本课程基于云化版本: Syslab 2024b 构建

本课程需要在首选项加载:

TyMachineLearning

学习本课程之前需要学习:

Syslab基本功能

Julia语法

课程示例的完整代码见附件-机器学习课程示例



CONTENTS

目录

Part 1

背景知识

Part 2

分类

Part 3

聚类

Part 4

降维

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

Part1

背景知识

1. 人工智能
2. 机器学习



@苏州同元软控信息技术有限公司版权所有
侵权必究

未经许可，不得复印、传播或以其他方式使用

1.人工智能-定义

人工智能 (Artificial Intelligence, AI) 是计算机科学的一个分支, 旨在使机器能够执行通常需要人类智能才能完成的任务。

具体来说, 就是让机器能够像人类一样思考和行动, 不仅旨在增强人类的脑力, 还包括增强体力。

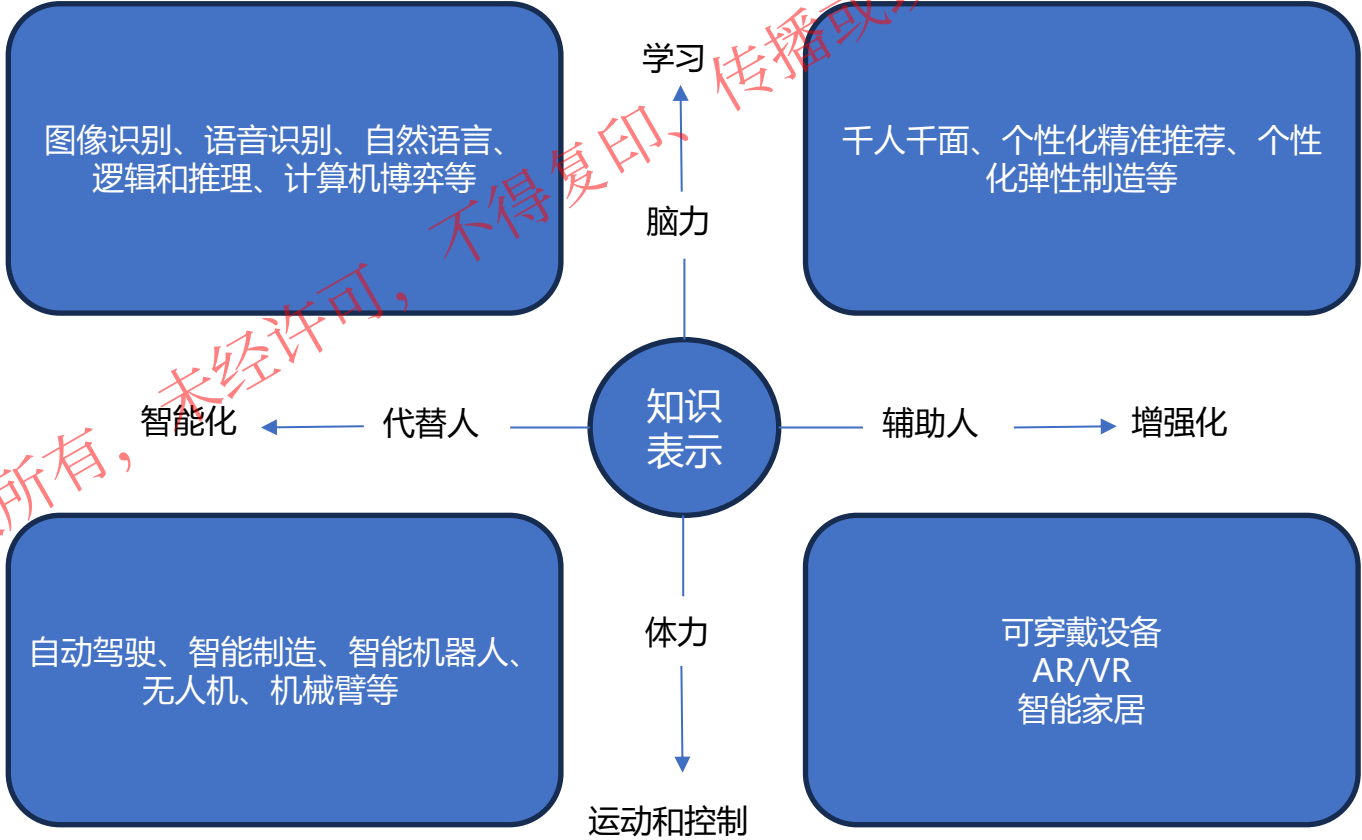
“人工智能是关于知识的学科, 怎样表示知识以及怎样获得知识并使用知识的科学。”

——Nils John Nilsson



1.人工智能-应用

人工智能作为现代科技发展的前沿领域，已经从理论研究走向了日常应用，训练完成的人工智能既能用于代替人完成任务，也可以用来辅助人达到目标。



2.机器学习-简介

01 机器学习的定义

机器学习，简而言之，是一种通过数据自动学习并改进的技术。它使计算机能够在不进行明确编程的情况下，从数据中提取模式、做出预测或决策。这种能力使得机器学习成为处理复杂、大规模数据集的有力工具。

02 机器学习的应用

机器学习技术已经广泛应用于各个领域，如金融、医疗、教育、交通、电商等。它不仅能够提高工作效率，还能为我们提供更加个性化、智能化的服务。例如，在金融领域，机器学习可以用于风险评估、欺诈检测和智能投顾等；在医疗领域，它可以辅助医生进行疾病诊断和药物研发等。



2.机器学习-核心要素

01 数据

数据是机器学习的基石。无论是训练模型还是评估性能，都离不开高质量的数据集。数据的数量、质量和多样性对机器学习的效果至关重要。

02 模型

模型是机器学习的核心。它代表了数据中的规律和模式，是机器学习算法输出的结果。选择合适的模型对于提高机器学习的性能至关重要。

03 算法

算法是机器学习的驱动力。它决定了如何根据数据训练模型、如何优化模型参数以及如何评估模型性能。不同的算法适用于不同的场景和问题。

机器学习就是通过**数据**驱动，以**算法**为工具，训练**模型**来揭示规律并完成任务的技术。

2.机器学习-三种范式区别

01 数据标注的差异

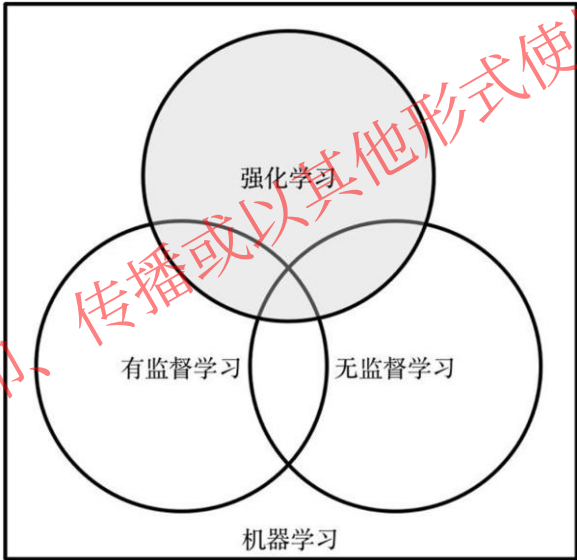
- 监督学习：需要有标签的输入数据（输入-输出对）。
- 无监督学习：仅需无标签的输入数据。
- 强化学习：数据来自与环境的交互，包括状态、动作和奖励。

02 目标的不同

- 监督学习：学习带标签的样本，目标是从已有的输入-输出对中学习映射关系，用于预测未知数据的输出。
- 无监督学习：学习未标注的数据，目标是发现数据的结构或分布规律，如聚类或降维。
- 强化学习：通过与环境交互，学习如何采取行动以最大化长期收益，目标是找到最佳策略。

03 学习过程的区别

- 监督学习：从训练集中拟合映射关系，评估时使用损失函数衡量预测误差。
- 无监督学习：通过优化目标函数（如聚类的紧密度）发现数据的潜在结构。
- 强化学习：通过试错法不断调整策略，根据奖励反馈优化行为。



Part2

分类

1. 基本概念
2. 算法介绍
3. 分类案例

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.基本概念

01 定义

分类是机器学习的一种监督学习任务，通过学习已有的带标签数据（训练集），构建一个模型，预测新数据所属的类别。

02 输入输出

输入：一组具有特定特征的数据。

输出：数据所属的离散类别。

03 常见任务

根据类别的数量，可以分为二分类与多分类

04 性能评估

常用指标：准确率（Accuracy）、精确率（Precision）、召回率（Recall）。

混淆矩阵：分析分类结果中不同类别的分布情况



2.算法介绍

分类算法的核心在于寻找一个决策边界，将数据空间划分成不同的区域，使每个区域对应一个类别，常见分类算法及原理如下：

算法	原理	适用场景
逻辑回归	基于线性模型，使用逻辑函数将线性回归扩展为概率分类	二分类任务，特征数量较少的数据
支持向量机（SVM）	寻找最大化类别间间隔的超平面，使用核函数扩展到非线性问题	文本分类、图像分类等高维数据问题
决策树	通过树状结构递归地分割数据空间，使用信息增益选择最优分裂点	医疗诊断、客户分群等规则清晰的任务
随机森林	集成多个决策树，通过投票提高模型鲁棒性和精度	各种分类任务的通用模型
朴素贝叶斯	基于条件概率公式，假设特征间独立性	图像分类、语音识别等复杂问题
K近邻（KNN）	基于实例的学习，通过距离度量找到最相似的 K 个邻居，进行投票分类	小数据集、实时分类任务

3.分类案例-树模型

在此介绍机器学习工具箱中树模型的使用方法。我们以泰坦尼克号的生存预测数据集为例子展开，讨论如何使用机器学习工具箱中的树模型处理实际的预测问题。

数据集介绍

Titanic数据集是一个关于 1912 年泰坦尼克号沉船事件中一些乘客的个人信息以及存活状况的数据集。Titanic 数据集主要包含两部分，训练集 (train.csv) 和测试集 (test.csv)。其中训练集中包含乘客的基本信息和最终在事故中的存活情况，测试集只包含乘客的基本信息，不包含存活情况。

Titanic 数据集主要由以下信息组成：

- survival: 乘客的存活情况，0表示遇难，1表示存活
- pclass: 舱位等级，1、2、3表示不同等级的舱位
- sex: 性别
- age: 年龄
- sibsp: 与该乘客一起上船的兄弟姐妹或者配偶的数量
- parch: 与该乘客一起上船的父母或者子女的数量
- ticket: 船票编号信息
- fare: 船票价格
- cabin: 舱位编号
- embarked: 上船地点，主要有3个，C=Cherbourg, Q=Queenstown, S=Southampton

3.分类案例-树模型

Titanic数据集可以用来进行数据分析和机器学习的练习，例如探索不同特征和存活率的关系，建立分类模型来预测乘客是否遇难等。本案例数据集中的特征包括：

Pclass : 舱位等级

Sex: 性别

Age: 年龄

Parch: 父母或子女数量

Fare: 船票价格

Embarked: 上船地点

Has_Cabin: 是否有居住船舱

FamilySize: 家庭成员数量

IsAlone: 是否孤身一人上船

Title: 头衔

- 本次的预测任务分为两个阶段：
 - 一阶段使用 3 种树模型对 Titanic 数据集分别进行预测
 - 二阶段整合一阶段的三组预测值堆叠作为特征值，再使用元学习模型（XGBoost）对堆叠预测值进行拟合，得到最后的预测模型
- 使用两阶段预测的堆叠树模型可以提高预测的准确性和健壮性。通过本案例，可以了解到如何使用机器学习工具箱中的树模型进行预测，同时，过程中还将使用到不同的绘图函数和统计计算函数。

3.分类案例-树模型

1.加载本案例需要使用的工具箱

```
using TyMachineLearning # 机器学习工具箱
using TyPlot # 图形工具箱
using CSV
using DataFrames
using TyStatistics
```

2.加载数据集，并对数据集进行手动划分

```
train_file =
joinpath(pkgdir(TyMachineLearning),
"data/Example/titanic_train.csv")
train_df = CSV.read(train_file,
DataFrame);
test_df = train_df[1:200, :];
train_df = train_df[201:end, :];

println(first(train_df, 5))
println(first(test_df, 5))
```

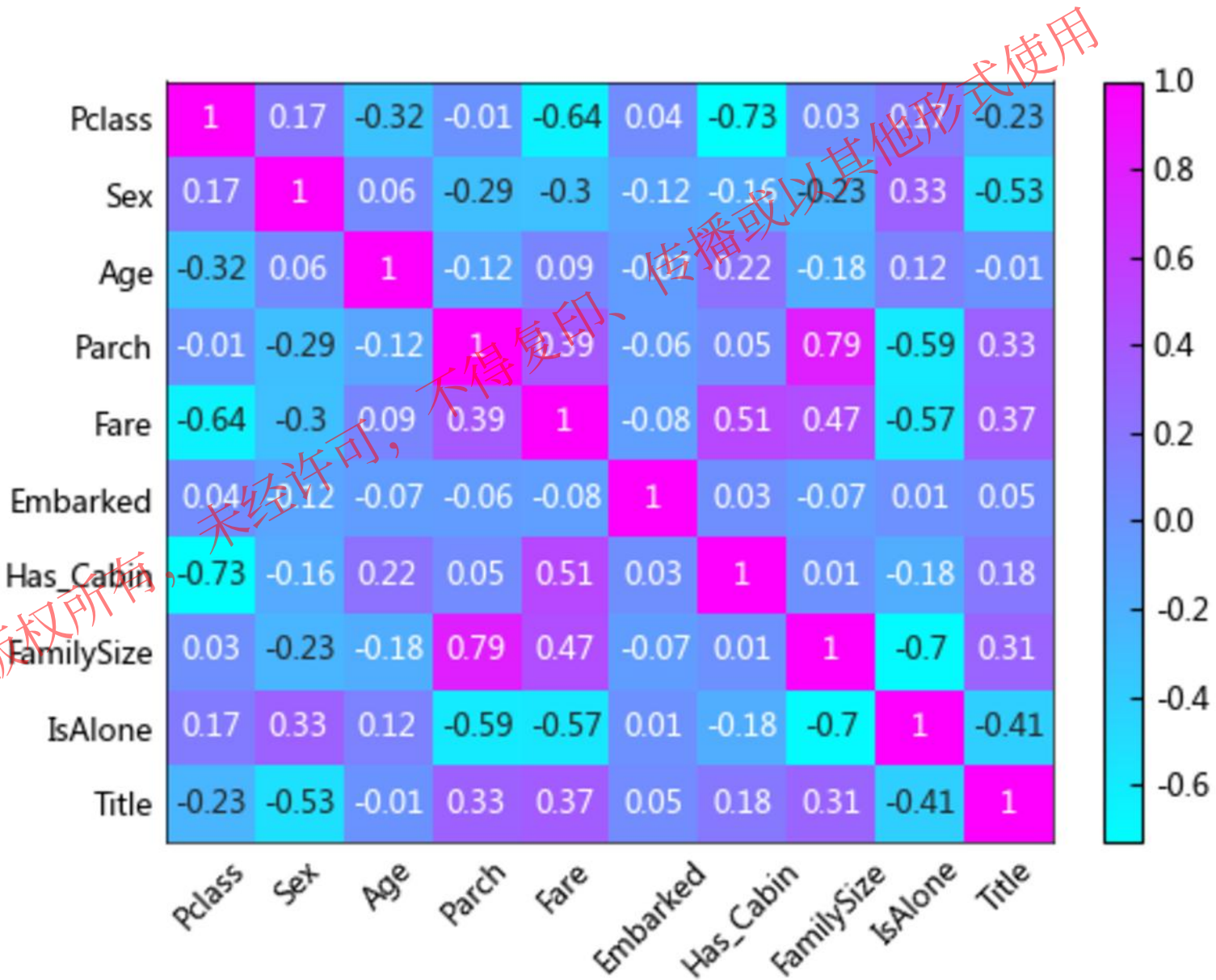
3.原始的测试集并没有真实的存活情况，因此我们直接从训练集中抽取一部分数据作为验证集，以检测模型的预测精度。接下来，我们对数据特征进行探索。

4.查看特征之间的相关性，我们使用统计工具箱中的相关性分析函数 corr() 对特征的相关性进行分析，并使用图形工具箱中的热力图函数heatmap() 对特征相关性进行可视化。

```
# 计算特征之间的相关性
features_names = names(train_df[:, 2:end]);
# 保存特征的名字，用于绘图
feature_corr, _ = corr(Matrix(train_df[:,
2:end])); # 计算特征之间的相关性
figure()
corr_heatmap = heatmap( # 绘制相关性热力图
    feature_corr; xvalues=features_names,
    yvalues=features_names, fmt=".2f", rotation =
    45)
tightlayout()
```

3.分类案例-树模型

通过相关性热力图我们发现：数据集中多数特征相关性不高。因此，我们可以认为多数特征之间一定程度上相互独立。



3.分类案例-树模型

本页代码为伪代码，运行可使用附件代码案例

接下来，我们可以建立模型，并对数据进行学习。

本案例使用 AdaBoost, LightGBM 和 RF 对数据集分别进行训练。经过树模型的训练，我们可以很方便地通过模型的训练结果，查看其中特征的重要性，以便找到数据中真正影响结果的特征。

```
# 训练数据准备
select!(train_df, Not(:Survived), :Survived);
select!(test_df, Not(:Survived), :Survived);
# 训练模型
ada_clf = AdaBoostTree(train_df, 500, 0.75, 9, 1);
lgbm_clf = GradientBoosting(
    Matrix(train_df[:, 1:(end - 1)]),
    train_df[:, end];
    n_estimators=500,
    max_depth=5,
    min_samples_leaf=2,
);
rf_clf = randomforest(
    Matrix(train_df[:, 1:(end - 1)]),
    train_df[:, end];
    n_estimators=500,
    max_depth=6,
    min_samples_leaf=2,
);
```

3.分类案例-树模型

查看其中特征的重要性:

```
ada-fi = ada-clf.feature_importances_;
lgbm-fi = lgbm-clf.feature_importances_;
rf-fi = rf-clf.feature_importances_;
ave-fi = (ada-fi + lgbm-fi + rf-fi) / 3;
println("AdaBoost: \t", ada-fi)
println("LGBM: \t\t", ada-fi)
println("Random Forest: \t", ada-fi)
println("平均: \t\t", ada-fi)
```

可视化特征重要性:

```
figure()
subplot(4, 1, 1)
scatter(features_names, ada-fi; filled=true)
xtickangle(12)
title("AdaBoost Feature Importances")
subplot(4, 1, 2)
scatter(features_names, lgbm-fi;
filled=true)
xtickangle(12)
title("LGBM Feature Importances")
subplot(4, 1, 3)
scatter(features_names, rf-fi; filled=true)
xtickangle(12)
title("Random Forest Feature Importances")
subplot(4, 1, 4)
scatter(features_names, ave-fi; filled=true)
xtickangle(12)
title("Average Feature Importances of 3
Models")
tightlayout()
```

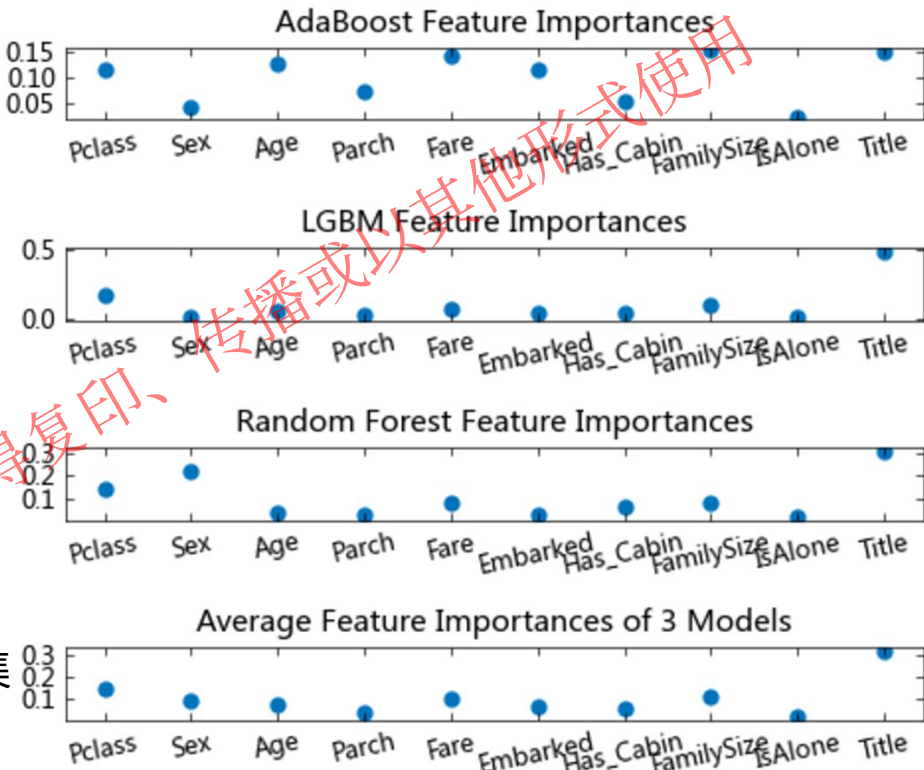
3.分类案例-树模型

经过我们的树模型训练，从中我们可以得到特征重要性排名。

- 在 AdaBoost 学习器中，家庭成员数量和头衔对是否存活影响较大；
- 在 LGBM 学习器中，头衔和舱位等级对是否存活影响较大；
- 在 Random Forest 学习器中，一个人的头衔和性别则是主要影响因素。
- 综合来看，在泰坦尼克号沉没事件中，一个人的头衔、舱位等级、家庭成员数量和性别很大程度上决定了这个人的存活与否。

通过训练模型，我们也可以对 Titanic 号上的人员进行存活预测，现在我们在验证集上评价我们模型的预测效果：

```
# 使用验证集查看各模型的预测精度
ada_score = ada_clf.score(Matrix(test_df[:, 1:(end - 1)]), test_df[:, end]);
lgbm_score = lgbm_clf.score(Matrix(test_df[:, 1:(end - 1)]), test_df[:, end]);
rf_score = rf_clf.score(Matrix(test_df[:, 1:(end - 1)]), test_df[:, end]);
print("AdaBoost: ", ada_score, "\n", "LGBM: ",
      lgbm_score, "\n", "RF: ", rf_score, "\n")
```



AdaBoost: 0.805
LGBM: 0.795
RF: 0.835

本页代码为伪代码，运行可使用附件代码案例

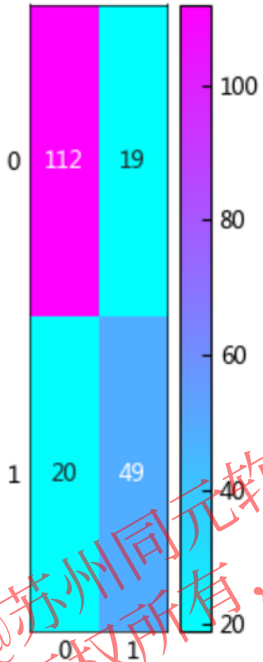


3.分类案例-树模型

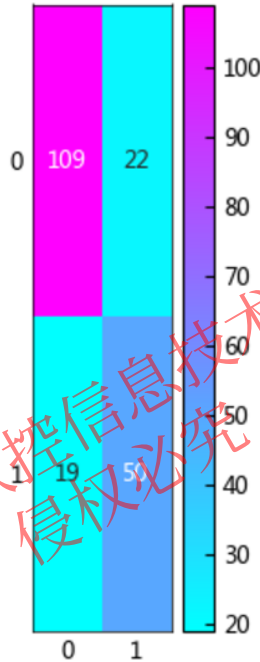
同时，我们可以创建一个函数计算预测的混淆矩阵，并查看混淆矩阵图。

```
figure()
ada_cm, ClassLabelsada = confusionmat(
    test_df[:, end],
    ada_clf.predict(Matrix(test_df[:, 1:(end -
1)])))
)
```

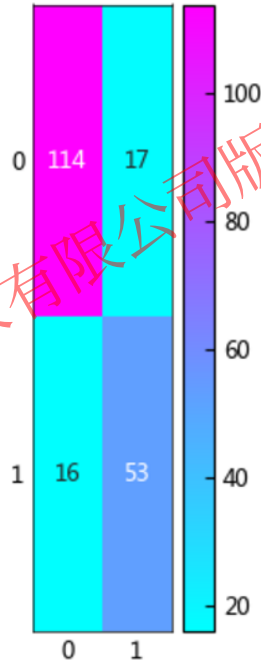
AdaBoost 混淆矩阵



LGBM 混淆矩阵



RF 混淆矩阵



```
subplot(1, 3, 1)
heatmap(ada_cm)
title("AdaBoost 混淆矩阵")

lgbm_cm, ClassLabelslgbm = confusionmat(
    test_df[:, end],
    lgbm_clf.predict(Matrix(test_df[:, 1:(end - 1)])))
)
```

```
subplot(1, 3, 2)
heatmap(lgbm_cm)
title("LGBM 混淆矩阵")
```

```
rf_cm, ClassLabelsrfr = confusionmat(
    test_df[:, end],
    rf_clf.predict(Matrix(test_df[:, 1:(end - 1)])))
)
```

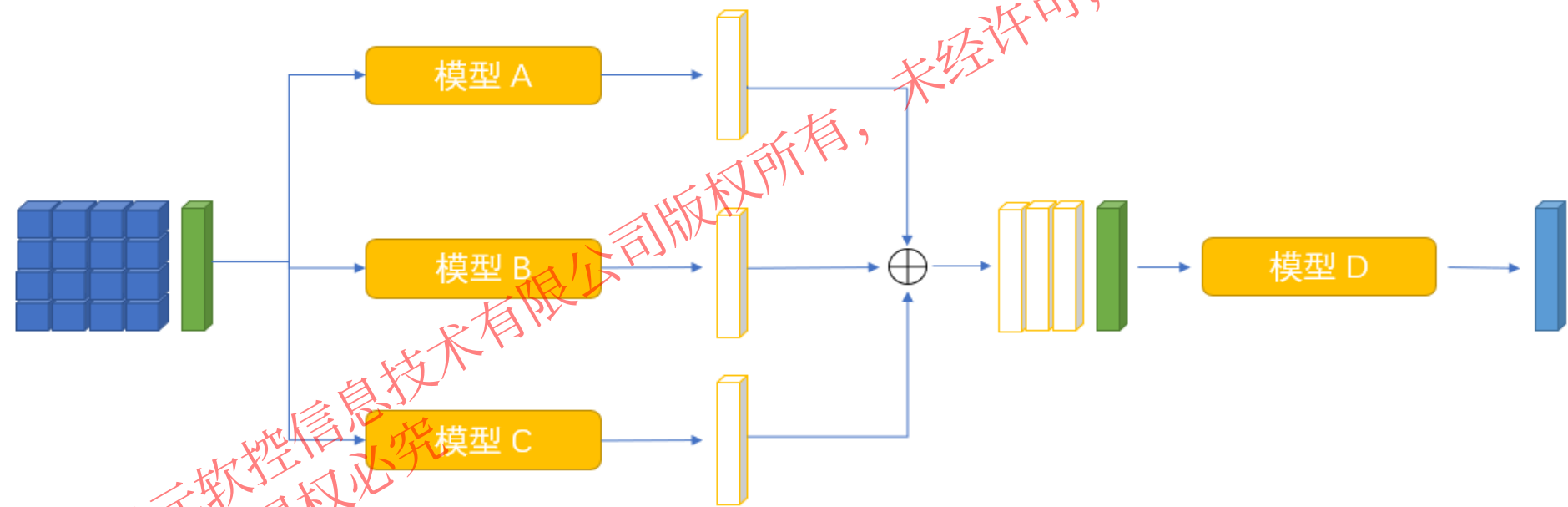
```
subplot(1, 3, 3)
heatmap(rf_cm)
title("RF 混淆矩阵")
```

3.分类案例-树模型

第二阶段我们对之前的 3 个树模型进行堆叠

使一阶段的预测结果作为二阶段的训练数据

最终得到一个具备健壮性的模型，具体的流程如下：



3.分类案例-树模型

我们使用一阶段的 3 个模型在训练集上进行预测，得到预测结果，并进行堆叠：

```
# 取出各模型的预测值，进行堆叠
ada_pred = ada_clf.predict(Matrix(train_df[:, 1:(end - 1)]))
lgbm_pred = lgbm_clf.predict(Matrix(train_df[:, 1:(end - 1)]))
rf_pred = rf_clf.predict(Matrix(train_df[:, 1:(end - 1)]))

stack_pred = DataFrame(; Ada_pred=ada_pred, lgbm_pred=lgbm_pred,
rf_pred=rf_pred)
first(stack_pred, 5)
```

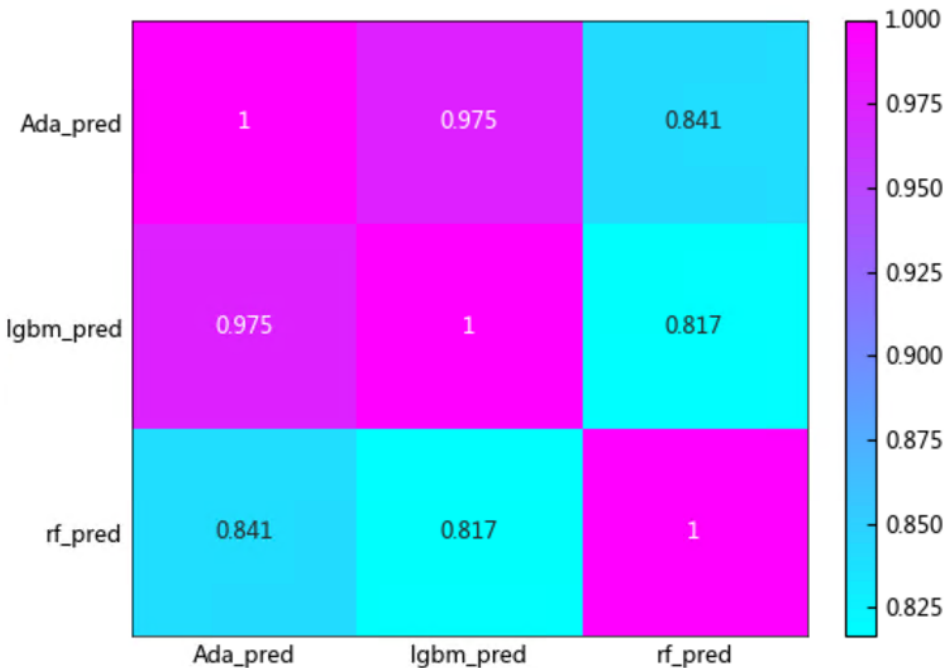
5x3 DataFrame

Row	Ada_pred Int32	lgbm_pred Int64	rf_pred glimpesd
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0
5	0	0	0

我们可以查看堆叠后的训练数据之间的相关性：

我们可以看到 AdaBoost 和 LGBM 之间的相关性较高。在实际的生产中，我们可以选用其他模型，使得各预测值之间的相关性尽可能小。

```
stack_names = names(stack_pred);
stack_corr, _ = corr(Matrix(stack_pred));
figure()
corr_heatmap = heatmap(stack_corr; xvalues=stack_names,
yvalues=stack_names, fmt=".3f")
```



3.分类案例-树模型

我们使用 XGBoost 训练堆叠数据：

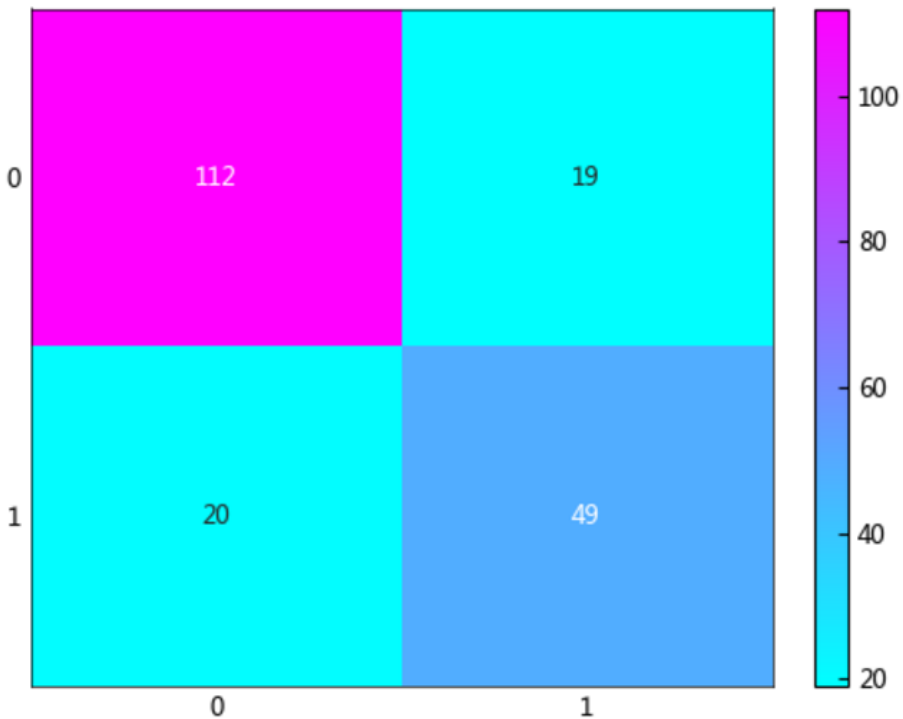
```
# 使用堆叠预测值训练一个 XGBoost 模型
xgb_clf = HistcGradientBoosting(Matrix(stack_pred), train_df[:, end])
```

训练了 3 个树模型对数据进行第一阶段的预测，再将第一阶段的预测值结合，进行第二阶段的预测。

有了 XGBoost 这个堆叠模型，我们在验证集上看看效果：

```
ada_test_pred = ada_clf.predict(Matrix(test_df[:, 1:(end - 1)]));
lgbm_test_pred = lgbm_clf.predict(Matrix(test_df[:, 1:(end - 1)]));
rf_test_pred = rf_clf.predict(Matrix(test_df[:, 1:(end - 1)]));
test_stack = DataFrame(
    Ada_test_pred=ada_test_pred, Lgbm_test_pred=lgbm_test_pred,
    Rf_test_pred=rf_test_pred
);
xgb_test_score = xgb_clf.score(Matrix(test_stack), test_df[:, end]);

print(xgb_test_score)
xgb_cm, ClassLabelsxgb = confusionmat(test_df[:, end],
xgb_clf.predict(Matrix(test_stack)))
figure()
heatmap(xgb_cm)
```



3.分类案例-树模型

通过混淆矩阵和评估效果，堆叠模型的验证集准确率达到 0.805。

一阶段和二阶段的准确率如下表所示：

模型	准确率
AdaBoost	0.805
LGBM	0.795
RF	0.835
堆叠 XGBoost	0.805

可以看到：

- 我们的堆叠模型的准确率高于一阶段中的 LGBM 模型，与AdaBoost 相当，比 Random Forest 低
- 从原理上分析，堆叠模型学习了一阶段中堆叠数据的模式，具备一定的容错率
- 健壮性要高于一阶段中的树模型

Part3

聚类

1. 基本概念
2. 算法介绍
3. 案例实践

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.基本概念

01 定义

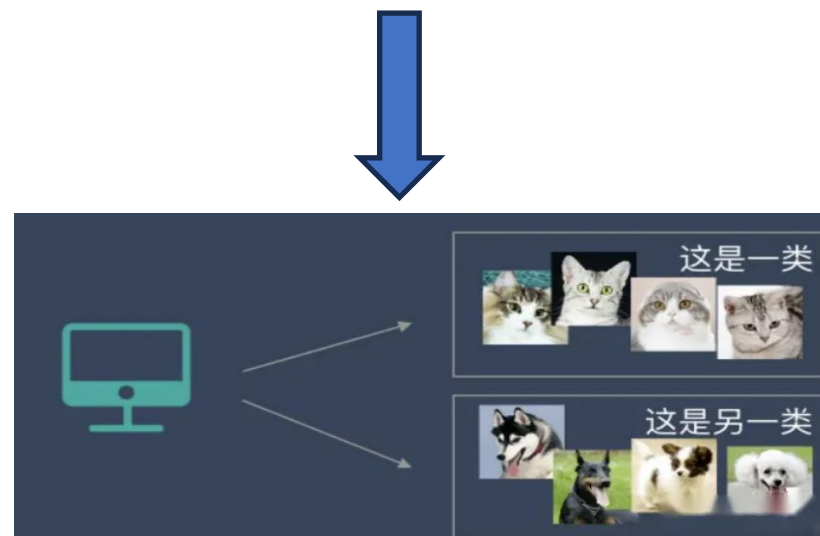
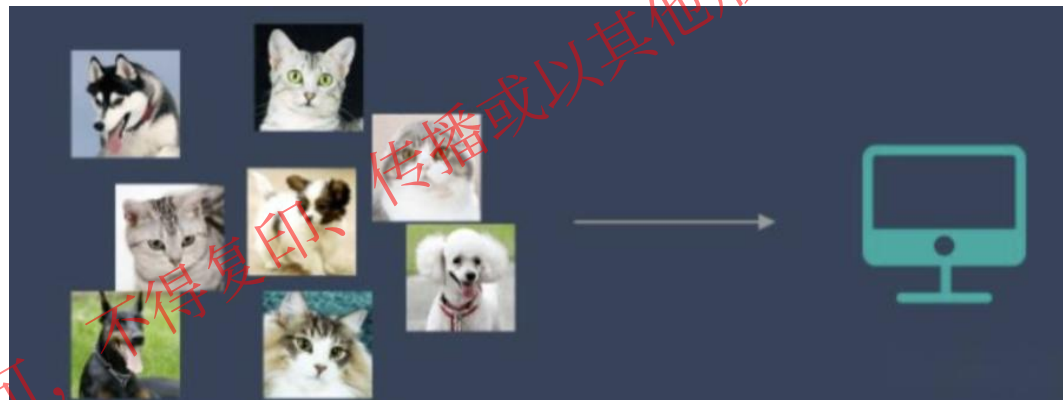
聚类是一种无监督学习方法，目的是将数据分组，使组内相似性最大，组间差异性最大。

02 核心思想

通过度量样本之间的相似性或距离，发现数据的内在结构和模式。

03 与分类的区别

分类需要有标签数据，属于有监督学习；聚类不依赖标签，属于无监督学习。



2. 算法介绍

聚类算法根据不同的假设和实现方式，可以分为以下几类：

01 基于划分的方法

通过迭代优化目标函数，将数据划分为若干组。

示例：K均值、K-中心点、PAM (Partitioning Around Medoids)

02 基于层次的方法

构建数据的层次结构，逐步合并或分裂聚类。

示例：层次聚类。

03 基于密度的方法

通过数据的密度分布找到聚类，能够发现任意形状的簇。

示例：DBSCAN、OPTICS。

04 基于模型的方法

假设数据由若干潜在模型生成，通过拟合这些模型找到聚类。

示例：高斯混合模型。

05 基于网格的方法

将数据空间划分为网格单元，根据密度对单元进行聚类。

示例：STING、CLIQUE。

2.算法介绍-层次聚类

01 定义

层次聚类是一种通过逐步合并或分裂样本点，建立层次关系并生成树状图的无监督学习方法，用于揭示数据内部的层次结构和相似性模式。

02 基本思想

层次聚类以样本间的相似性为基础，逐步构建数据的层次结构，最终以树状结构（称为树状图）表示聚类结果。主要分为以下两种策略

01 凝聚型聚类

流程：从每个样本作为单独的簇开始，逐步将相似的簇合并，直到所有样本归为一个簇或达到预定条件。

步骤：计算所有样本对之间的相似性（距离矩阵）。找到最相似的两个簇，合并成一个簇。

更新距离矩阵，重新计算新簇与其他簇的距离。

重复上述步骤，直至所有样本合并成一个簇。

02 分裂型聚类

流程：从所有样本作为一个整体开始，通过递归地分裂簇，最终将每个样本单独分开

步骤：将当前簇划分为两个较小的簇，使分裂后簇内相似性最大化。重复划分，直到每个样本成为单独的簇。

3.案例实践

fp = cluster(X, n)返回使用矩阵X中的层次聚类后的模型结果列表

通过指定聚类数进行层次聚类

在加载的数据上执行聚合聚类，在不同层次对数据集进行划分，从而形成树形的聚类结构

```
# 加载鸢尾花数据集
using TyMachineLearning
using CSV
using DataFrames
using TyPlot
file = joinpath(pkgdir(TyMachineLearning),
"data/Classification/iris.csv")
iris = CSV.read(file, DataFrame; header=false)
X = iris[:, 1:4]
```



山鸢尾(*Iris setosa*)



变色鸢尾(*Iris versicolor*)



维吉尼亚鸢尾(*iris virginica*)

分类特征：花萼（sepal）和花瓣（petal）的宽度和长度

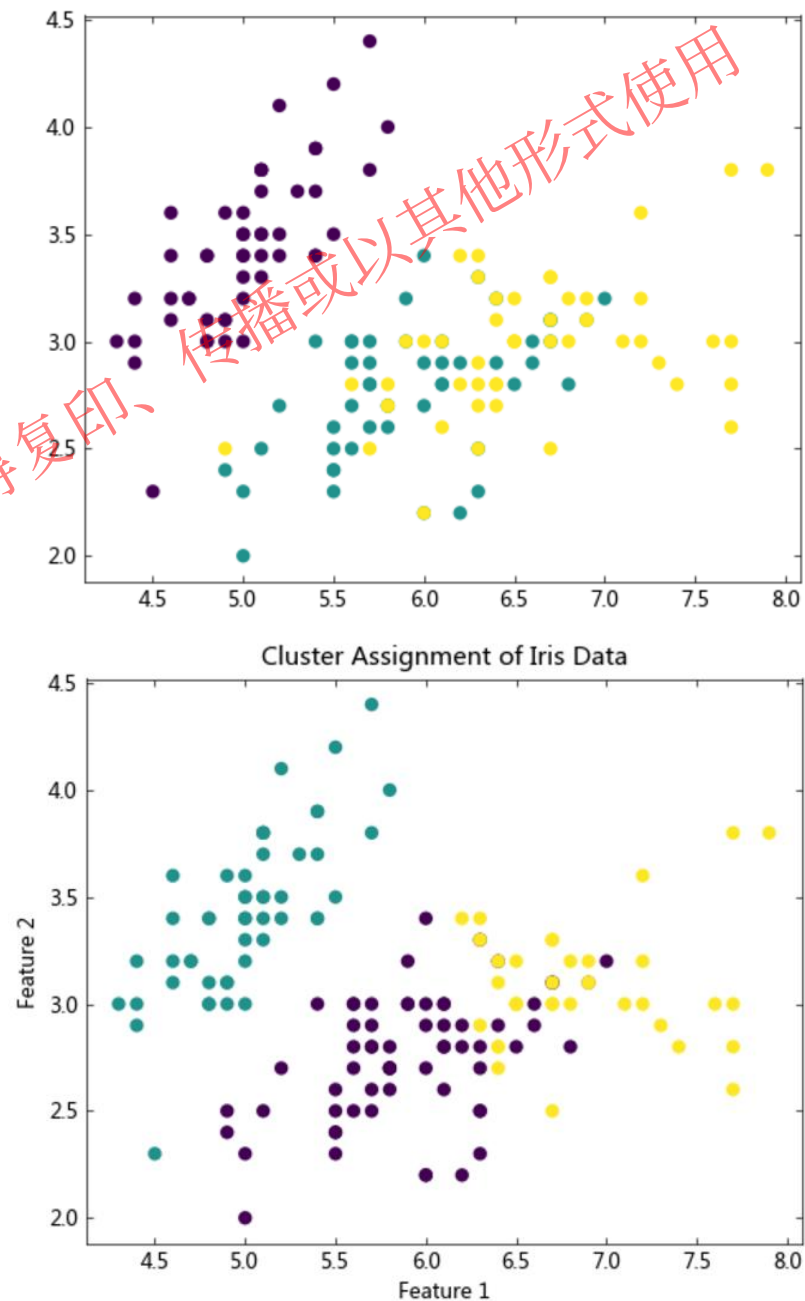
3.案例实践

```
# 可视化原始数据集
rename!(iris, [:Column5 => :SH])
X1,y1,convs=convent_columns(iris,"SH")
fgdata_x1=Matrix(X1.values)
fgdata_y1=Array(y1.values)
scatter(X[:,1],X[:,2];c=fgdata_y1,filled="true")
```

```
# 对输入数据X训练层次聚类模型
n = 3 #聚类数
clf, idx = cluster(fgdata_x1, n)
# 可视化层次聚类后对数据的分配情况
scatter(X[:, 1], X[:, 2]; c=Array(idx),
filled="true")

xlabel("Feature 1")
ylabel("Feature 2")
title("Cluster Assignment of Iris Data")
```

本页代码为伪代码，运行可使用附件代码案例



Part4

降维

1. 基本概念
2. 算法介绍
3. 案例实践



@苏州同元软控信息技术有限公司版权所有, 侵权必究

1.基本概念

01 定义

降维是指通过减少特征数量，简化数据表示，同时尽可能保留信息。

02 意义

降低计算复杂度，提高模型效率。消除冗余或噪声特征，提升模型性能。更直观地可视化高维数据。

03 应用场景

数据可视化：将高维数据投影到二维/三维空间，便于理解和分析。

数据压缩：通过降维减少特征数量，从而降低存储和传输成本，同时保留数据的主要信息。

数据预处理：降维作为一种数据预处理技术，为其他任务（如分类、聚类）提供低维但信息丰富的输入。

2.算法介绍

01 局部线性嵌入 Locally Linear Embedding

假设高维数据在低维流形上，可以用局部线性结构表示。LLE通过保持局部线性重构权重，将高维数据映射到低维空间。即每个点在高维空间中的局部表示方式在低维空间中仍然成立。

Standard LLE（标准LLE）：原始的LLE算法，依赖局部线性权重保持低维数据的流形结构。适合简单的流形结构，容易受到噪声影响。

Modified LLE（改进型LLE）：引入正则化项以改善权重矩阵的稳定性，能够更好地处理噪声和高维稀疏数据。

Hessian LLE (Hessian-based LLE)：通过引入 Hessian 矩阵来捕捉数据的局部曲率信息，从而在降维时更好地保留高维数据的非线性结构。

LTSA (Local Tangent Space Alignment)：基于局部切空间对齐的LLE变体。通过计算局部切空间并对齐这些空间以保持流形结构的全局一致性。

2.算法介绍

02 等度量映射嵌入 Isomap Embedding

基于流形假设，将高维数据投影到低维空间，同时保留数据点之间的测地距离（沿流形表面计算的距离）。使用最短路径算法估计高维空间中数据点之间的测地距离，并通过经典MDS完成降维。

03 多维尺度变换 Multidimensional Scaling (MDS)

MDS是一种降维方法，通过最小化原始高维数据点之间的距离与低维空间中点之间的距离的差异来实现。主要用于将数据点的距离矩阵转化为几何表示。

03 非线性降维的谱嵌入 Spectral Embedding for Non-linear Dimensionality Reduction

利用图论方法，将数据点建模为图，边的权重代表数据点之间的相似性。通过计算图的拉普拉斯矩阵特征值，将数据嵌入到低维空间，同时保留邻近点的关系。

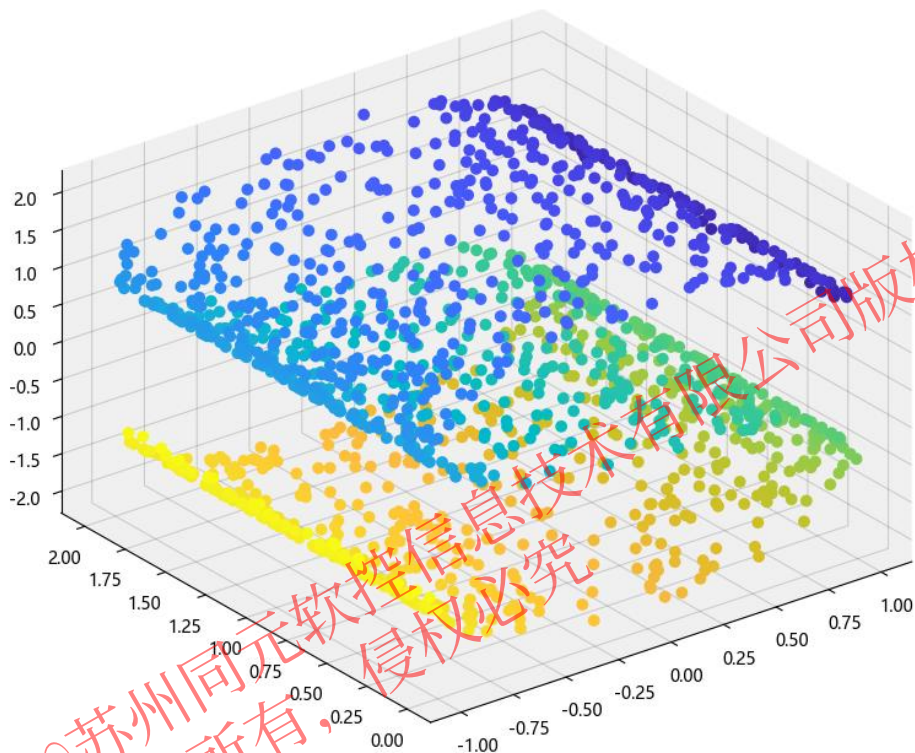
04 t 分布随机邻域嵌入 T-distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE是一种非线性降维方法，特别适合高维数据的可视化。它通过将高维空间中点对的相似性分布与低维空间中点对的相似性分布进行匹配，最小化两者之间的KL散度。

4. 案例实践

以 S 形曲面数据集为例子，展示 Syslab 机器学习工具箱降维功能

- 使用流形学习算法
- 对数据降维，同时保留数据点之间的关系
- 达到展开数据集的效果



涉及的算法包括：

- Locally Linear Embeddings
 - Standard
 - LTSA
 - Hessian
 - Modified
- Isomap Embedding
- Multidimensional scaling
- Spectral embedding for non-linear dimensionality reduction
- T-distributed Stochastic Neighbor Embedding

4. 案例实践

我们的数据集呈现出一个 S 型曲面的形状，其中的数据点在不同的地方有着不一样的密度分布，我们将使用非线性降维的方法；

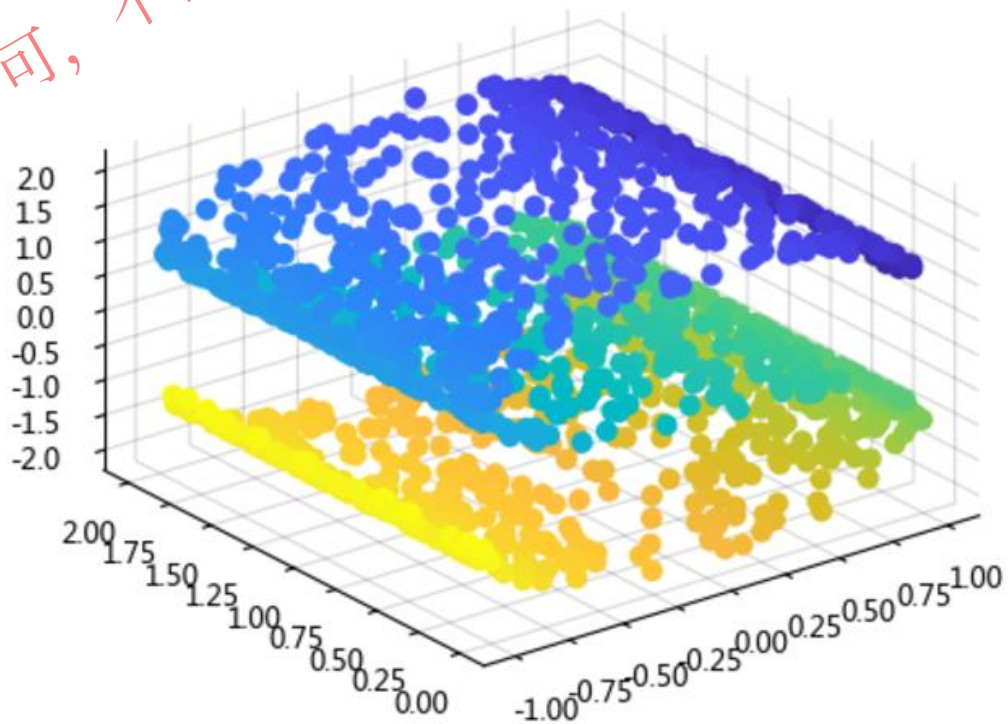
非线性降维的核心思想是：高维数据常常嵌套在一个低维流形中，表面上的高维特征可能是冗余的。通过揭示数据的低维结构，我们可以更好地进行可视化或后续分析。

- 从机器学习工具箱中导入 S Curve 数据集

```
using TyMachineLearning
S_points, S_color = get_curve() # 获取数据集
```

- 使用图形工具箱对数据集进行可视化：

```
using TyPlot
scatter3(S_points[:, 1], S_points[:, 2],
S_points[:, 3], c=S_color, s=50, filled=true)
```



4. 案例实践

绘制的图形较多，为了更方便地进行绘图，我们定义一个函数来调用图形工具箱内的 scatter 二维散点图工具，来简化绘图的过程：

```
"""
plot_2d(data, fig, fig_title): 使用 TyPlot 绘制降维后的 2D 散点图
    args:
        data: 用于绘图的 2D 数据
        fig: 用于绘图的画布
        fig_title: 图标题
"""

function plot_2d(data, fig, fig_title; sub_pos=(0, 0, 0))
    if sub_pos != (0, 0, 0)
        subplot(sub_pos[1], sub_pos[2], sub_pos[3])
    end
    figure(fig)
    scatter(data[:, 1], data[:, 2]; c=S_color, s=50, filled=true)
    title(fig_title)
    return true
end
```

4. 案例实践

局部线性嵌入（LLE）可以被认为是一系列局部主成分分析，它们在全局进行比较，以找到最佳的非线性嵌入。

本例中将要测试 4 种不同的 LLE 模型。

我们首先定义一个函数来定义这 4 种模型，该函数返回一个字典，该字典包括了 4 种模型计算的不同结果：

```
function build_lle_models(data)
    # 定义一个函数来构建并训练 4 种 LLE 模型
    print("-> Computing standard...\n")
    sd_embedding, ev = fitLocallyLinearEmbedding(data;
n_neighbors=12, eigen_solver="auto")
    lle_standard = sd_embedding.embedding_
    print("-> Computing ltssa...\n")
    lle_ltssa =
fitltssaEmbedding(data).fit_transform(data)
    print("-> Computing hessian...\n")
    lle_hessian =
fitHessianEigenmapping(data).fit_transform(data)
    print("-> Computing modified...\n")
    lle_mod = fitMLLEmbedding().fit_transform(data)
    return Dict(
        "lle_standard" => lle_standard,
        "lle_ltssa" => lle_ltssa,
        "lle_hessian" => lle_hessian,
        "lle_mod" => lle_mod,
    )
end
```

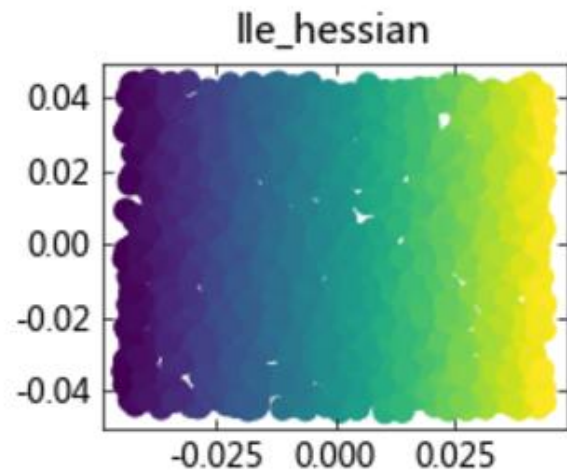
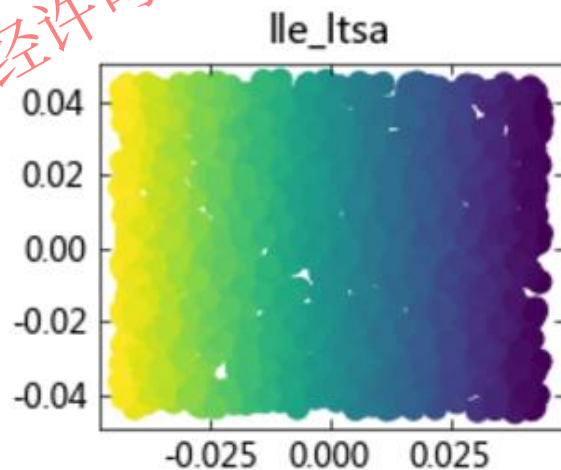
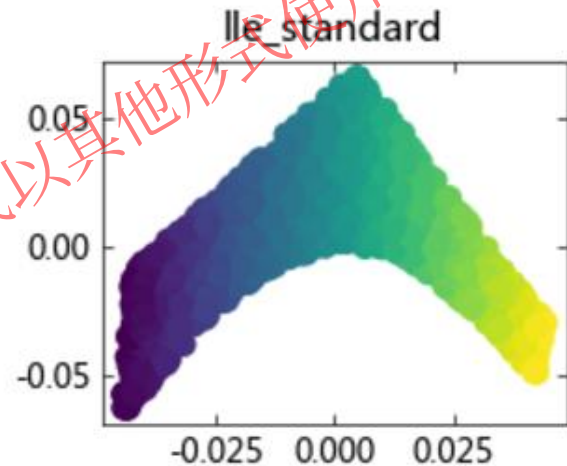
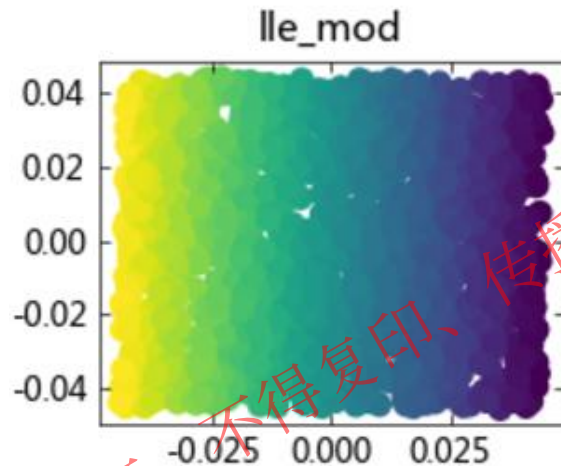
4. 案例实践

调用这个函数，得到 4 个 LLE 模型的计算结果：

```
# 使用 LLE 模型对数据进行降维
lle_models = build_lle_models(S_points)
```

得到计算结果后，使用前面我们定义的 plot_2d 函数绘制图像，对计算结果进行可视化。此处我们使用循环结构，对结果字典中的各个模型结果进行遍历，再在每次循环中调用 plot_2d 函数绘制各个模型的可视化结果：

```
fig = figure()
global position = 0 # 用于确定子图位置
for key in keys(lle_models)
    global position += 1
    plot_2d(lle_models[key], fig, key;
    sub_pos=(2, 2, position))
end
tightlayout()
```



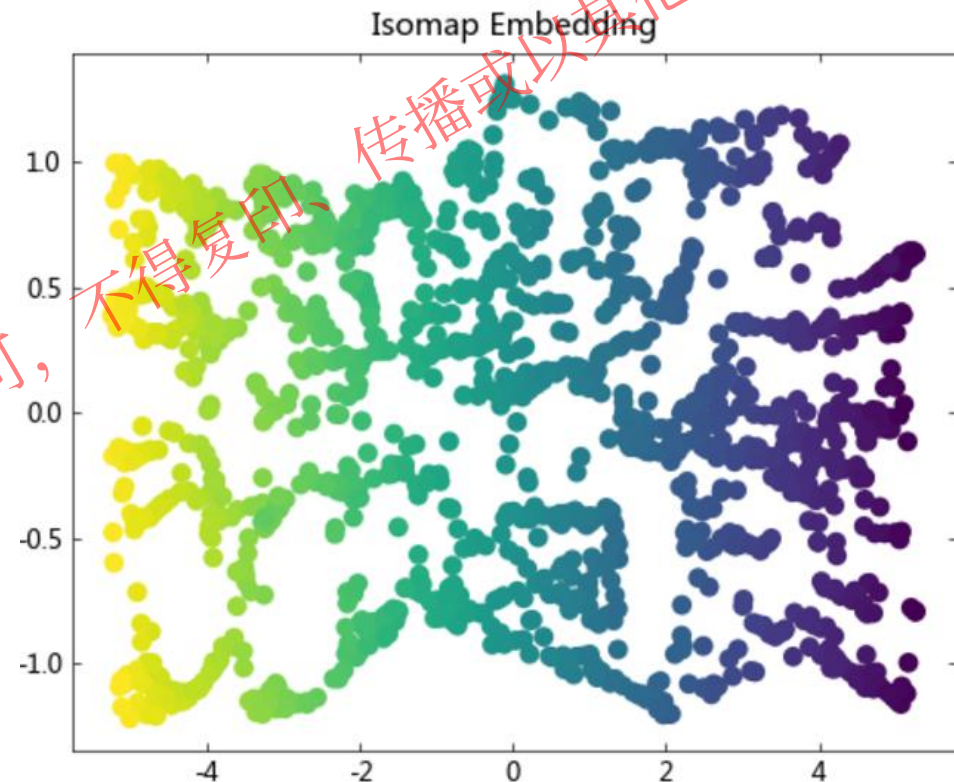
4. 案例实践

通过等距映射进行非线性降维。

Isomap 寻求一个低维嵌入，保持所有点之间的测地距离。

以下，使用机器学习工具箱中的 Isomap 模型对数据进行降维：

```
# 使用 Isomap 模型对数据进行降维
print("-> Computing isomap...\n")
isomap_embedding, isomap_ev =
fitIsomap(S_points)
isomap_result =
isomap_embedding.embedding_
plot_2d(isomap_result, figure(), "Isomap
Embedding")
```



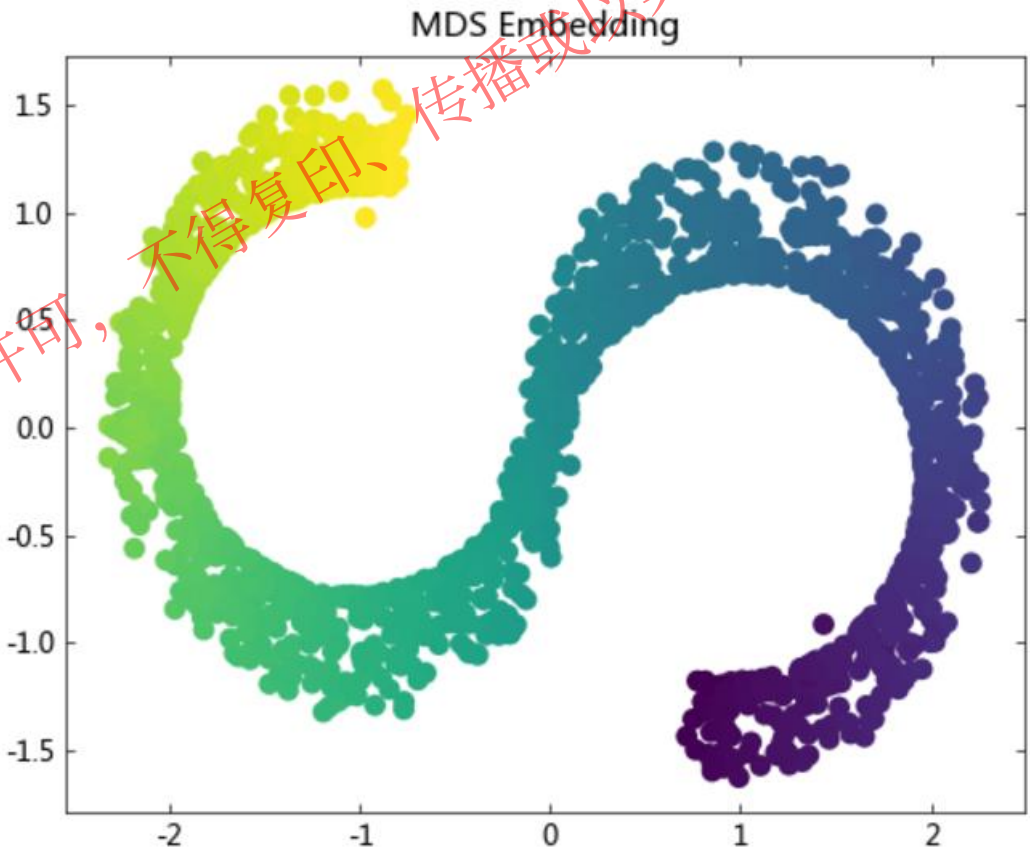
4.案例实践

多维缩放（MDS）寻求数据的低维表示，使得距离尽可能地保持原始高维空间中的距离。

通常，MDS是一种用于分析相似性或差异性数据的技术。

```
# 使用 MDS 模型对数据进行降维
print("-> Computing MDS...\n")
mds_embedding, ts = cmdscale(S_points)
mds_result =
mds_embedding.fit_transform(S_points)

plot_2d(mds_result, figure(), "MDS
Embedding")
```

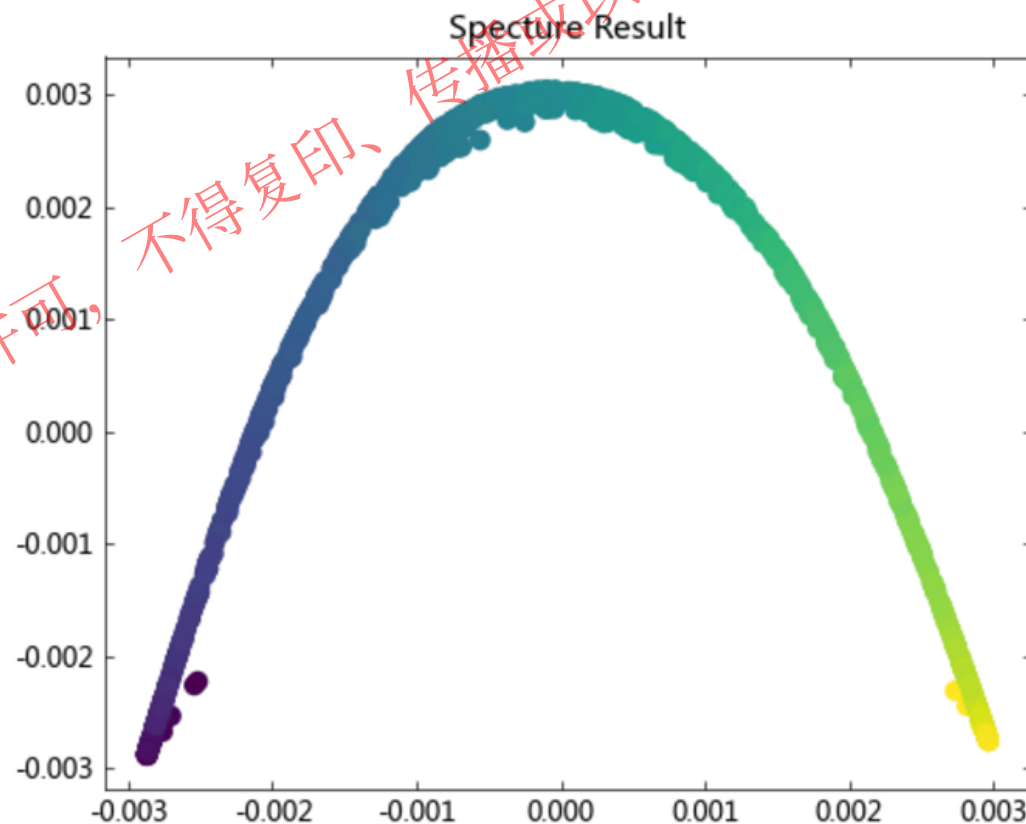


4. 案例实践

非线性降维的谱嵌入

实现使用拉普拉斯特征映射，它通过对图拉普拉斯算子进行谱分解，找到数据的低维表示。

```
# 使用 Spectral 模型对数据进行降维
print("-> Computing spectral...\n")
spectral_embedding =
fitSpectralEmbedding("nearest_neighbors"
)
spectral_result =
spectral_embedding.fit_transform(S_point
s)
plot_2d(spectral_result, figure(),
"Spectre Result")
```



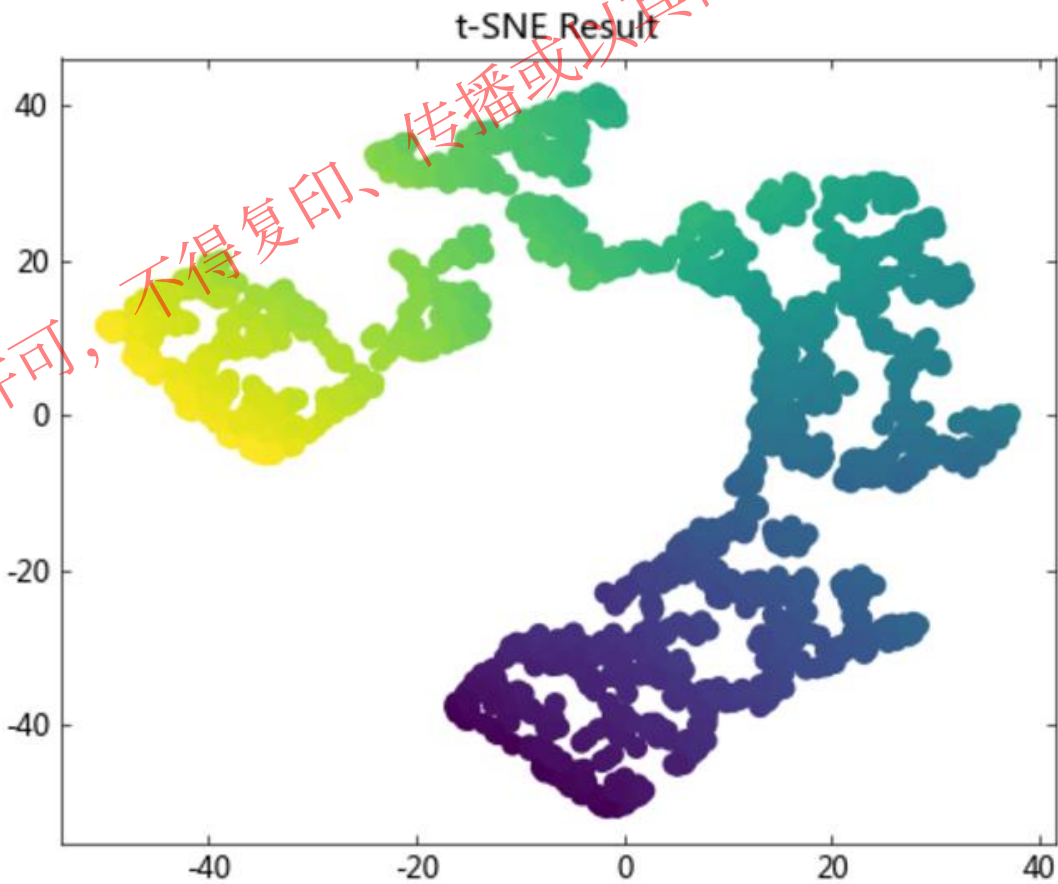
4.案例实践

t 分布随机邻域嵌入将数据点之间的相似性转换为联合概率，
最小化低维嵌入和高维数据的联合概率之间的 Kullback-Leibler 散度。
t-SNE 有一个非凸的代价函数，即不同的初始化会得到不同的结果。

```
# 使用 tsne 模型对数据进行降维
print("-> Computing tsne...\n")
tsne_embedding = tsne(; init="random")
tsne_result =
tsne_embedding.fit_transform(S_points)

plot_2d(tsne_result, figure(), "t-SNE
Result")
```

由于函数运行过程中存在随机性，因此上述代码运行结果可能略有差异。



感谢您的聆听!

Thanks



未经授权，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究